

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct *top-down and bottom up* parsers using *CFG* for parsing conflicts and error recovery for efficient syntax free generation. rsrcs

Assessment Tool weightage: 70%

QUESTIONS:

Total Marks:50

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I V.Vamsidhar Reddy 192365029 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the
student:

V. Vamsidhar Reddy

Annexure A

SCENARIO-BASED TASK

QUESTIONS:

- Scenario: Parsing Table Conflict in LL(1) While designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input symbol combination. This leads to confusion during parsing. Analyze how the syntax analyzer uses FIRST and FOLLOW sets in this process, identify the reason for the conflict, and explain how it affects deterministic parsing. Conclude by suggesting what decisions should be taken to modify the grammar so that it becomes suitable for LL(1) parsing.
- Scenario: Incorrect Parse Tree Generation A syntax analyzer generates a parse tree for the expression $id + id$, but the evaluation gives incorrect results due to improper grouping of operators. Analyze how the syntax analyzer constructs the parse tree, identify the issue related to associativity, and explain how grammar rules influence the structure of the tree. Finally, discuss what decisions should be taken to enforce correct associativity and ensure accurate parsing.
- Scenario: Invalid Expression Handling A syntax analyzer in a compiler receives the token sequence corresponding to the expression $a + b$ from the lexical analyzer. Although all tokens are valid individually, the parser fails to generate a parse tree and reports an error. In this situation, analyze how the syntax analyzer identifies the error based on grammar rules, determine the key issue in the token arrangement, and explain how parsing techniques such as predictive or shift-reduce parsing handle such cases. Further, discuss what decisions should be taken to improve error detection and recovery while maintaining clarity in parsing.
- Scenario: Ambiguity in Conditional Statements A syntax analyzer is designed using the grammar $S \rightarrow E \mid E \mid E \mid E \mid E$. While constructing the parsing table, conflicts arise, and the parser cannot decide which production to apply for certain inputs. In this scenario, explain how the syntax analyzer encounters ambiguity, identify the key issue known as the dangling else problem, and discuss how it impacts parsing. Suggest an appropriate modification or parsing strategy to resolve the ambiguity and ensure correct association of else clauses.
- Scenario: Operator Precedence Conflict A parser is designed for arithmetic expressions using the grammar $E \rightarrow E + E \mid E * E \mid id$. While parsing the input $id + id$, the syntax analyzer produces an incorrect parse tree due to ambiguity in operator precedence. Examine how the syntax analyzer

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding within the gaps	Partial understanding, misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues, some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decision is made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

NAME:- V. Vamsidhar Reddy - 192365029

COURSE:- CSA1402 - Compiler Design

→ CO2 - AT3 - SCENARIO BASED TASK

① Parsing Table conflict in LL(1):

In an LL(1) parser, the parsing table is constructed using the FIRST and FOLLOW set of the grammar.

For a production such as:

$A \rightarrow a$

The parser uses the $FIRST(a)$ set to determine which input symbol can begin the production. Suppose two productions.

$A \rightarrow a$ is $FIRST(a) \cap FIRST(B) = \phi$
 $A \rightarrow B$

For example,

$A \rightarrow id +$
 $A \rightarrow a \dots$
 $\uparrow$
Conflict

Another possible conflict occurs when one production can derive ϵ . In that case, FIRST and FOLLOW is also satisfy the LL(1) conditions.

(2) Incorrect parse tree generation:

* The syntax analyzer constructs a parse by applying production according to the input tokens. For the expression $id + id - id$. The parse tree determinants the order in which operation are the evaluated.

* If the grammar does not specify operator associativity. For example, It means may be interpreted $(id + (id - id))$ instead of the correct. It may the incorrect result.

Grammar rules directly influence the structure of the parse tree. A grammar that does not enforce associativity allows multiple valid parse tree for the same expression.

Decision; The grammar should enforce left associativity allows multiple for addition and subtraction.

for example,

This ensures expression are evaluated left to right

$$E \rightarrow E + T / E - T / T$$

$$T \rightarrow id.$$

③ Invalid Expression Handling:

* the lexical analyzer correctly identifies the tokens in the expression of $a * b$ as identifier and operators. However, the syntax analyzer checks whether the token follows the grammar rules.

* the error occurs because the grammar expects an operand after the '+' operator but another '*' appears instead. Therefore, the tokens sequence violates the grammar.

In predictive parsing (LL), the parser detects the error when no valid production exists in the parsing table for current input symbol. In shift level reduce parsing (LR) the parser detects the error when no valid shift or reduce action.

Decision:-

The parser should implement error detection and recovery techniques such as panic mode recovery, phrase level recovery or synchronization using Follow sets.

④ Ambiguity in Conditional Statements:

The grammar

* $S \rightarrow \text{if } E \text{ then } S$

* $S \rightarrow \text{if } E \text{ then } S \text{ else } S$

* $S \rightarrow a$

Is ambiguous because the parser cannot determine which if statement an else belongs to. This is known as the dangling else problem.

When constructing the parsing table, conflicts arise because both productions begin with same $\text{if } E \text{ then } S$.

The parser cannot decide whether to reduce using the first production or continue parsing the second production continues.

Decision:-

The grammar should be rewritten to distinguish matched and unmatched.

$S \rightarrow M \mid u$

$M \rightarrow \text{if } E \text{ then } M \text{ else } M/a$

$u \rightarrow \text{if } E \text{ then } S \text{ / if } E \text{ then } M \text{ else } u.$

⑤ Operator Precedence Conflict.

The grammar;

$$\bullet E \rightarrow E + E$$

$$\bullet E \rightarrow E * E$$

$$\bullet E \rightarrow id$$

It does not define operator precedence or associativity.

For an input $id + id * id$, two different parse trees.

$$\underline{*} (id + id) * id$$

$$\underline{*} id + (id * id)$$

The correct evaluation requires multiplication to have higher precedence than addition. Since the grammar,

Decision:- The grammar should be rewritten to enforce precedence

for example, $E \rightarrow E + T / T$

$$T \rightarrow T * F / F$$

$$F \rightarrow id$$

Hence,

* multiplication has higher precedence than Addition.

* Addition and multiplication are the left associative.